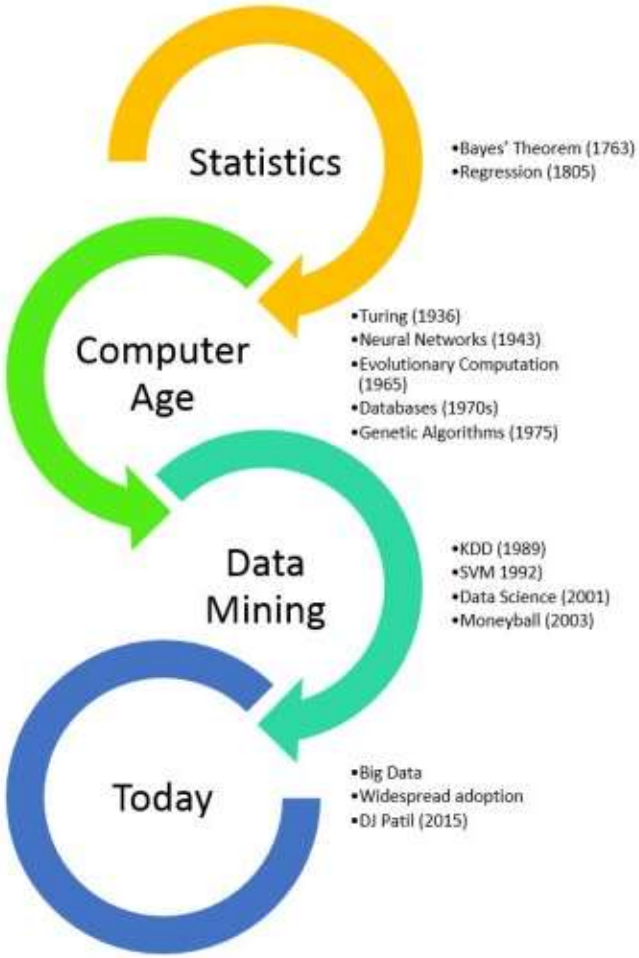
Feature Engineering (Feature Importance and Selection)

Dr. Tran Anh Tuan,

Faculty of Mathematics and Computer Science,

University of Science, HCMC



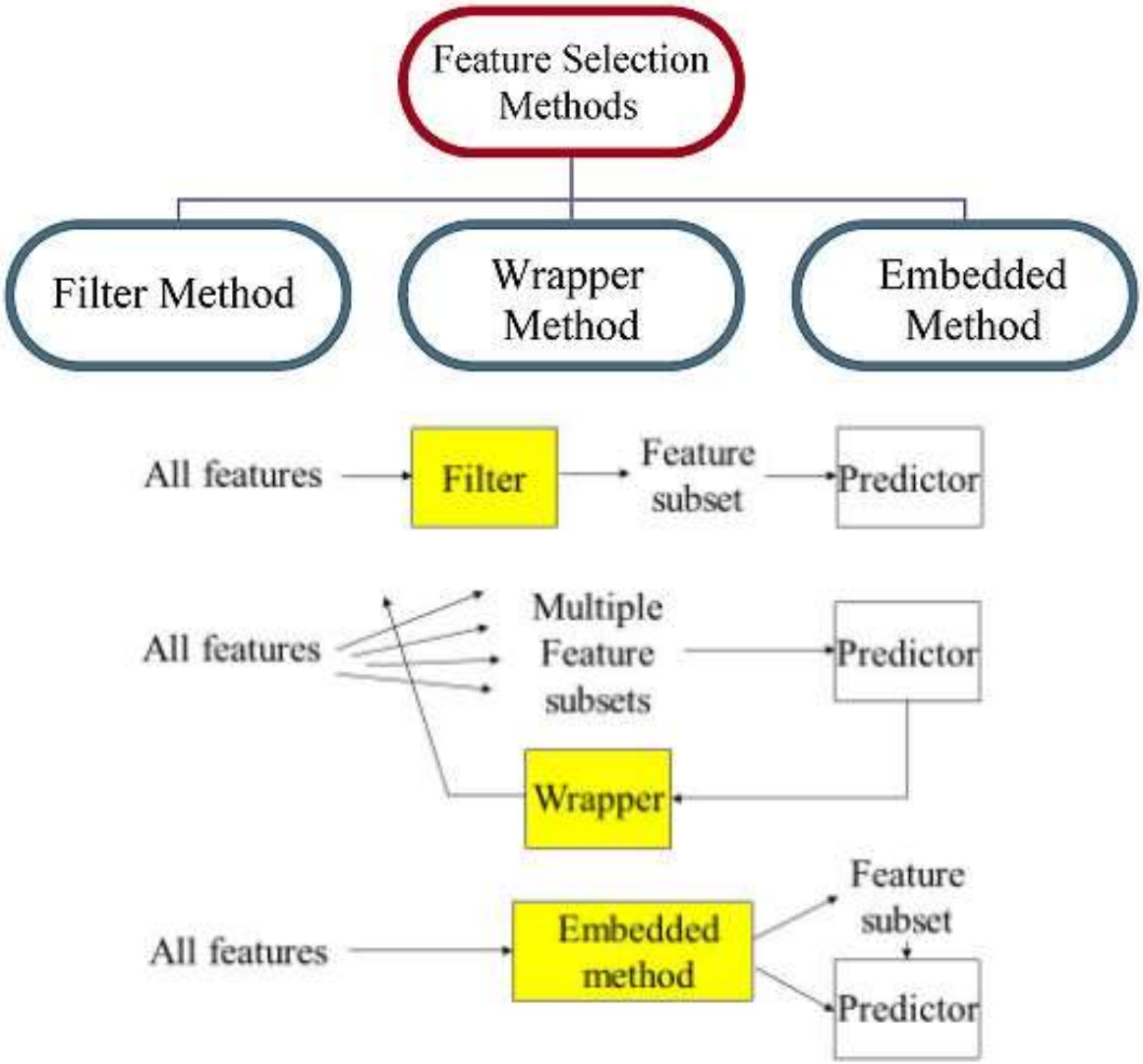
Evolutionary Step	Business Question	Enabling Technologies	Product Providers	Characteristics
Data Collection (1960s)	"What was my total revenue in the last five years?"	Computers, tapes, disks	IBM, CDC	Retrospective, static data delivery
Data Access (1980s)	"What were unit sales in New England last March?"	Relational databases (RDBMS), Structured Query Language (SQL), ODBC	Oracle, Sybase, Informix, IBM, Microsoft	Retrospective, dynamic data delivery at record level
Data Warehousing & Decision Support (1990s)	"What were unit sales in New England last March? Drill down to Boston."	On-line analytic processing (OLAP), multidimensional databases, data warehouses	Pilot, Comshare, Arbor, Cognos, Microstrategy	Retrospective, dynamic data delivery at multiple levels
Data Mining (Emerging Today)	"What's likely to happen to Boston unit sales next month? Why?"	Advanced algorithms, multiprocessor computers, massive databases	Pilot, Lockheed, IBM, SGI, numerous startups (nascent industry)	Prospective, proactive information delivery

Syllabus

- Lecture 1 : Data Preprocessing
- Lecture 2 : Explanatory Data Analysis
- **Lecture 3 : Feature Engineering (Feature Importance and Selection)**
- Lecture 4 : Association Rule Learning
- Lecture 5 : Unsupervised Clustering
- Lecture 6 : Unsupervised Clustering (cont.)
- Lecture 7 : Anomaly and Outlier Detection
- Lecture 8 : Regression and Classification Learning
- Lecture 9 : Recommendation Learning
- Lecture 10 : Final Project Requirement

Content

Content


Feature Selection Methods
Filter Method
Wrapper Method
Embedded Method
All features
Filter
Feature subset
Predictor
All features
Multiple Feature subsets
Predictor
Wrapper
All features
Embedded method
Feature subset
Predictor

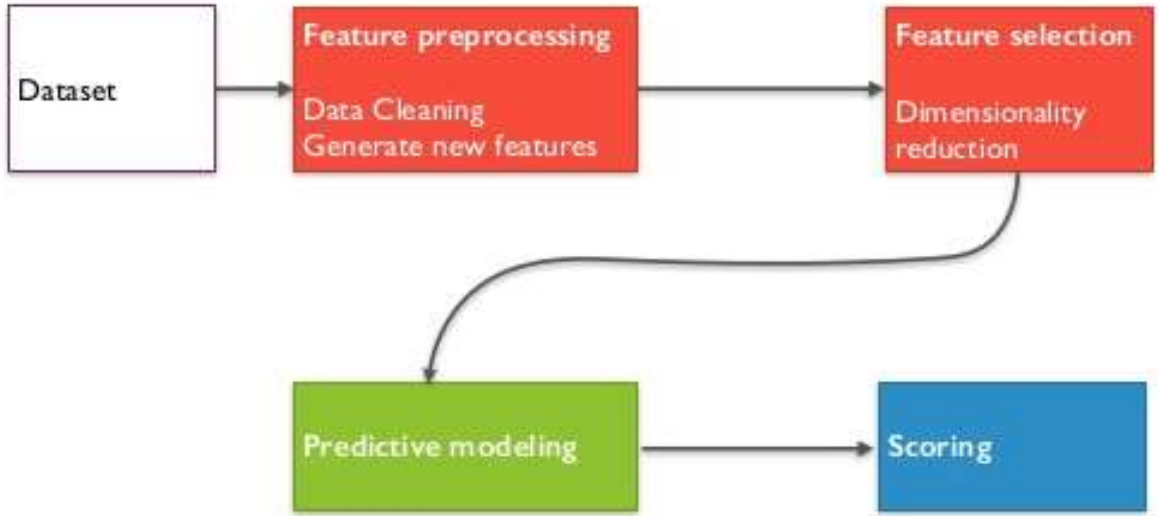
Importance of Feature Selection

- Machine learning works on a simple rule – if you put garbage in, you will only get garbage to come out
- This becomes even more important when the number of features are very large. You need not use every feature at your disposal for creating an algorithm. You can assist your algorithm by feeding in only those features that are really important.

Importance of Feature Selection

Top reasons to use feature selection are:

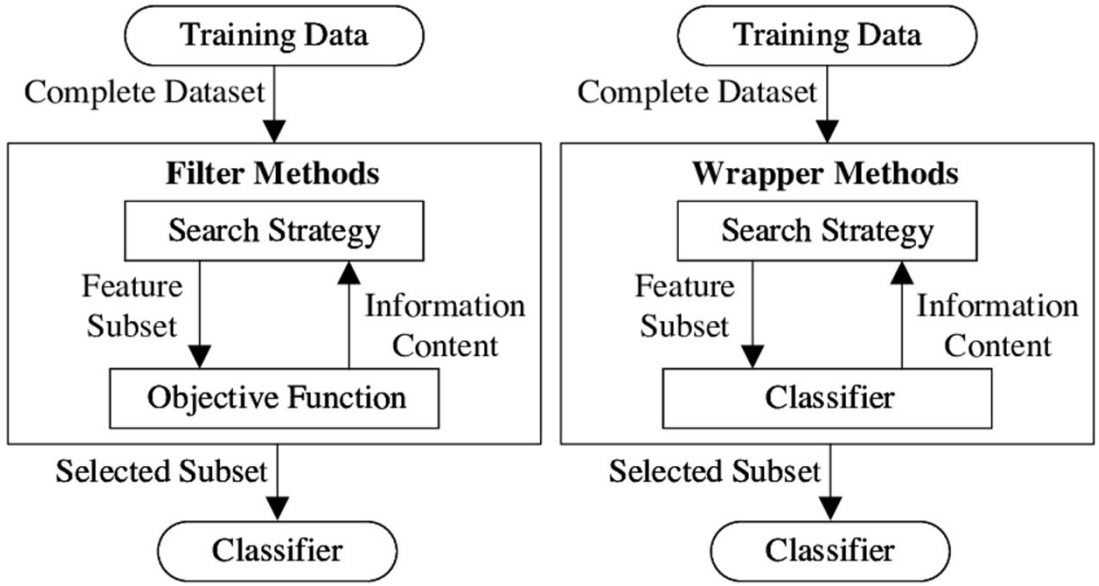
- It enables the machine learning algorithm to train faster.
- It reduces the complexity of a model and makes it easier to interpret.
- It improves the accuracy of a model if the right subset is chosen.
- It reduces overfitting.



Filter Methods

Filters methods belong to the category of feature selection methods that select features independently of the machine learning algorithm model. This is one of the biggest advantages of filter methods.

Features selected using filter methods can be used as an input to any machine learning models. Another advantage of filter methods is that they are very fast. Filter methods are generally the first step in any feature selection pipeline.



Filter Methods

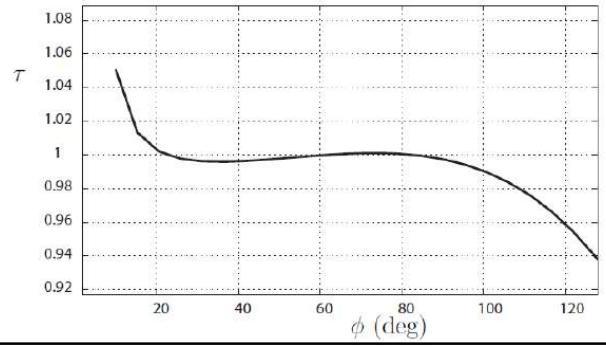
- **Missing Value Ratio**
- Suppose you're given a dataset. What would be your first step? You would naturally want to explore the data first before building model. While exploring the data, you find that your dataset has some missing values. Now what? You will try to find out the reason for these missing values and then impute them or drop the variables entirely which have missing values (using appropriate methods).
- What if we have too many missing values (say more than 50%)? Should we impute the missing values or drop the variable? I would prefer to drop the variable since it will not have much information. However, this isn't set in stone. We can set a threshold value and if the percentage of missing values in any variable is more than that threshold, we will drop the variable.

Complete the ratio table.

4	3
	6
12	9
16	12
20	15

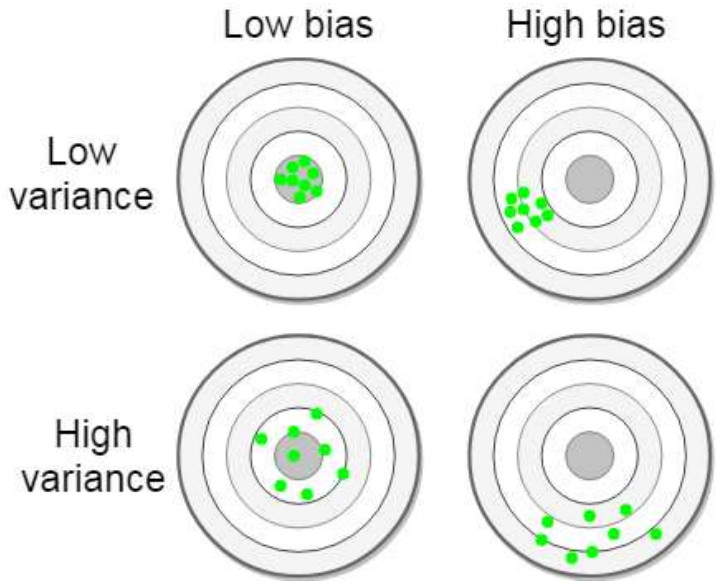
Filter Methods

- **Removing Constant features**
- Constant features are the type of features that contain only one value for all the outputs in the dataset. Constant features provide no information that can help in classification of the record at hand. Therefore, it is advisable to remove all the constant features from the dataset.
- **Removing Quasi-Constant features**
- Quasi-constant features, as the name suggests, are the features that are almost constant. In other words, these features have the same values for a very large subset of the outputs. Such features are not very useful for making predictions. There is no rule as to what should be the threshold for the variance of quasi-constant features. However, as a rule of thumb, remove those quasi-constant features that have more than 99% similar values for the output observations.



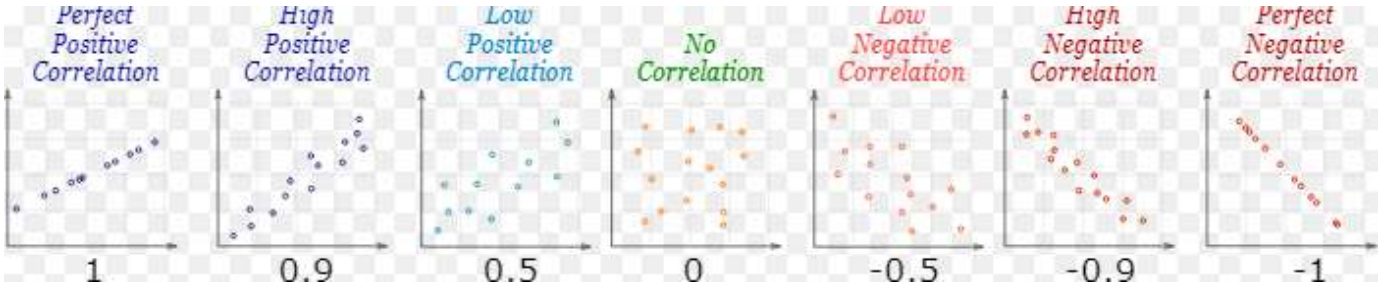
Filter Methods

- **Low Variance Filter**
- Consider a variable in our dataset where all the observations have the same value, say 1. If we use this variable, do you think it can improve the model we will build? The answer is no, because this variable will have zero variance.
- So, we need to calculate the variance of each variable we are given. Then drop the variables having low variance as compared to other variables in our dataset. The reason for doing this, as I mentioned above, is that variables with a low variance will not affect the target variable.



Filter Methods

- **High Correlation filter**
- High correlation between two variables means they have similar trends and are likely to carry similar information. This can bring down the performance of some models drastically (linear and logistic regression models, for instance). We can calculate the correlation between independent numerical variables that are numerical in nature. If the correlation coefficient crosses a certain threshold value, we can drop one of the variables (dropping a variable is highly subjective and should always be done keeping the domain in mind).
- **As a general guideline, we should keep those variables which show a decent or high correlation with the target variable.**



Filter Methods

Feature\Response	Continuous	Categorical
Continuous	Pearson's Correlation	LDA
Categorical	Anova	Chi-Square

- **Pearson's Correlation:** It is used as a measure for quantifying linear dependence between two continuous variables X and Y. Its value varies from -1 to +1. Pearson's correlation is given as:

$$\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y}$$

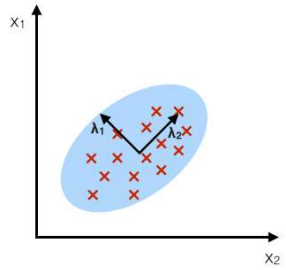
- **LDA:** Linear discriminant analysis is used to find a linear combination of features that characterizes or separates two or more classes (or levels) of a categorical variable.

- **ANOVA:** ANOVA stands for Analysis of variance. It is similar to LDA except for the fact that it is operated using one or more categorical independent features and one continuous dependent feature. It provides a statistical test of whether the means of several groups are equal or not.

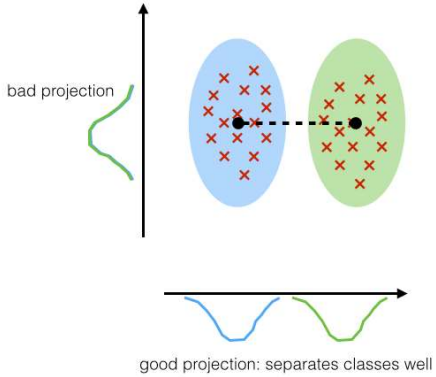
- **Chi-Square:** It is a is a statistical test applied to the groups of categorical features to evaluate the likelihood of correlation or association between them using their frequency distribution.

Filter Methods

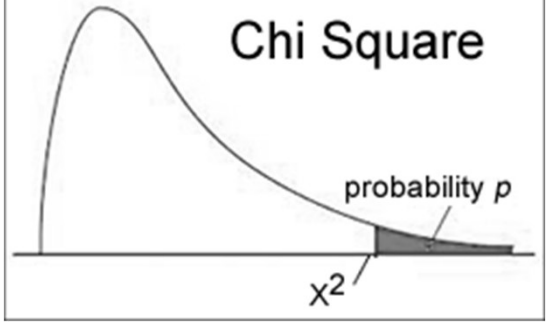
PCA:
component axes that maximize the variance



LDA:
maximizing the component axes for class-separation

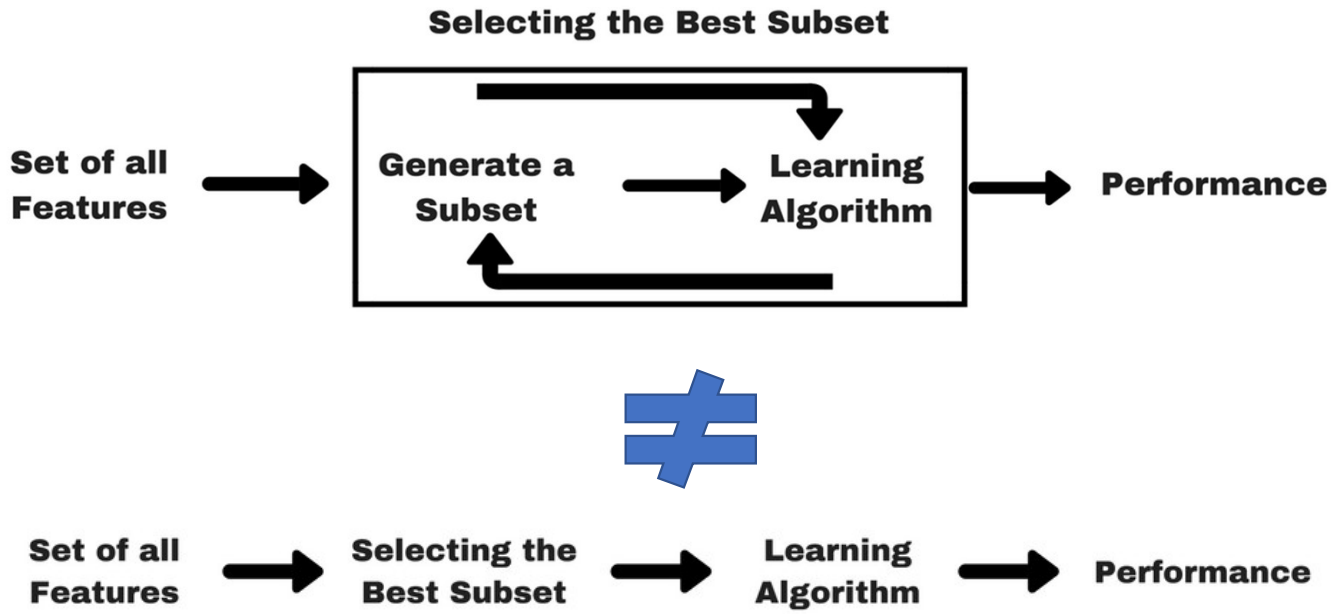


$$\chi_c^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$



Wrapper Methods

- In wrapper methods, we try to use a subset of features and train a model using them. Based on the inferences that we draw from the previous model, we decide to add or remove features from your subset. The problem is essentially reduced to a search problem. These methods are usually computationally very expensive.



Wrapper Methods

- Some common examples of wrapper methods are forward feature selection, backward feature elimination, recursive feature elimination, etc.
- **Forward Selection:** Forward selection is an iterative method in which we start with having no feature in the model. In each iteration, we keep adding the feature which best improves our model till an addition of a new variable does not improve the performance of the model.
- **Backward Elimination:** In backward elimination, we start with all the features and removes the least significant feature at each iteration which improves the performance of the model. We repeat this until no improvement is observed on removal of features.
- **Recursive Feature elimination:** It is a greedy optimization algorithm which aims to find the best performing feature subset. It repeatedly creates models and keeps aside the best or the worst performing feature at each iteration. It constructs the next model with the left features until all the features are exhausted. It then ranks the features based on the order of their elimination.

Wrapper Methods

Forward Feature Selection

- This is the opposite process of the Backward Feature Elimination we saw above. Instead of eliminating features, we try to find the best features which improve the performance of the model. This technique works as follows:
- We start with a single feature. Essentially, we train the model n number of times using each feature separately
- The variable giving the best performance is selected as the starting variable
- Then we repeat this process and add one variable at a time. The variable that produces the highest increase in performance is retained
- We repeat this process until no significant improvement is seen in the model's performance

Wrapper Methods

Forward Feature Selection

Sequential Forward Selection (SFS)

Input: $Y = \{y_1, y_2, \ldots, y_d\}$

- The ***SFS*** algorithm takes the whole d-dimensional feature set as input.

Output: $X_k = \{x_j \mid j = 1, 2, \ldots, k;\ x_j \in Y\}$, where $k = (0, 1, 2, \ldots, d)$

- SFS returns a subset of features; the number of selected features k, where $k < d$, has to be specified *a priori*.

Initialization: $X_0 = \emptyset,\ k = 0$

- We initialize the algorithm with an empty set $\emptyset$ ("null set") so that $k = 0$ (where k is the size of the subset).

Wrapper Methods

Forward Feature Selection

Step 1 (Inclusion):

$$x^+ = \arg\max J(x_k + x), \text{ where } x \in Y - X_k$$
$$X_{k+1} = X_k + x^+$$
$$k = k + 1$$

Go to Step 1

- in this step, we add an additional feature, x^+, to our feature subset X_k.
- x^+ is the feature that maximizes our criterion function, that is, the feature that is associated with the best classifier performance if it is added to X_k.
- We repeat this procedure until the termination criterion is satisfied.

Termination: $k = p$

- We add features from the feature subset X_k until the feature subset of size k contains the number of desired features p that we specified *a priori*.

Wrapper Methods

Forward Feature Selection

Sequential Forward Floating Selection (SFFS)

Input: the set of all features, $Y = \{y_1, y_2, \ldots, y_d\}$

- The ***SFFS*** algorithm takes the whole feature set as input, if our feature space consists of, e.g. 10, if our feature space consists of 10 dimensions (***d = 10***).

Output: a subset of features, $X_k = \{x_j \mid j = 1, 2, \ldots, k;\ x_j \in Y\}$, where $k = (0, 1, 2, \ldots, d)$

- The returned output of the algorithm is a subset of the feature space of a specified size. E.g., a subset of 5 features from a 10-dimensional feature space (***k = 5, d = 10***).

Initialization: $X_0 = Y, k = d$

- We initialize the algorithm with an empty set ("null set") so that the ***k = 0*** (where ***k*** is the size of the subset)

Wrapper Methods

Forward Feature Selection

Step 1 (Inclusion):

$$x^+ = \arg\max J(x_k + x), \text{ where } x \in Y - X_k$$
$$X_{k+1} = X_k + x^+$$
$$k = k + 1$$

Go to Step 2

Wrapper Methods

Forward Feature Selection

Step 2 (Conditional Exclusion):

$$x^- = \arg\max J(x_k - x), \text{ where } x \in X_k$$
$$if\ J(x_k - x) > J(x_k - x):$$
$$X_{k-1} = X_k - x^-$$
$$k = k - 1$$

Go to Step 1

- In step 1, we include the feature from the ***feature space*** that leads to the best performance increase for our ***feature subset*** (assessed by the ***criterion function***). Then, we go over to step 2
- In step 2, we only remove a feature if the resulting subset would gain an increase in performance. If $k = 2$ or an improvement cannot be made (i.e., such feature x^+ cannot be found), go back to step 1; else, repeat this step.
- Steps 1 and 2 are repeated until the **Termination** criterion is reached.

Termination: stop when ***k*** equals the number of desired features

Wrapper Methods

Backward Feature Elimination

Follow the below steps to understand and use the 'Backward Feature Elimination' technique:

- We first take all the n variables present in our dataset and train the model using them
- We then calculate the performance of the model
- Now, we compute the performance of the model after eliminating each variable (n times), i.e., we drop one variable every time and train the model on the remaining n-1 variables
- We identify the variable whose removal has produced the smallest (or no) change in the performance of the model, and then drop that variable
- Repeat this process until no variable can be dropped

Backward Feature Elimination

Wrapper Methods

Sequential Backward Selection (SBS)

Input: the set of all features, $Y = \{y_1, y_2, \ldots, y_d\}$

- The SBS algorithm takes the whole feature set as input.

Output: $X_k = \{x_j \mid j = 1, 2, \ldots, k;\ x_j \in Y\}$, where $k = (0, 1, 2, \ldots, d)$

- SBS returns a subset of features; the number of selected features k, where $k < d$, has to be specified *a priori*.

Initialization: $X_0 = Y$, $k = d$

- We initialize the algorithm with the given feature set so that the $k = d$.

Backward Feature Elimination

Wrapper Methods

Step 1 (Exclusion):

$$x^- = \arg\max J(x_k - x), \text{ where } x \in X_k$$
$$X_{k-1} = X_k - x^-$$
$$k = k - 1$$

Go to Step 1

- In this step, we remove a feature, x^- from our feature subset X_k.
- x^- is the feature that maximizes our criterion function upon re,oval, that is, the feature that is associated with the best classifier performance if it is removed from X_k.
- We repeat this procedure until the termination criterion is satisfied.

Termination: $k = p$

- We add features from the feature subset X_k until the feature subset of size k contains the number of desired features p that we specified *a priori*.

Backward Feature Elimination

Wrapper Methods

Sequential Backward Floating Selection (SBFS)

Input: the set of all features, $Y = \{y_1, y_2, \ldots, y_d\}$

- The SBFS algorithm takes the whole feature set as input.

Output: $X_k = \{x_j \mid j = 1, 2, \ldots, k;\ x_j \in Y\}$, where $k = (0, 1, 2, \ldots, d)$

- SBFS returns a subset of features; the number of selected features k, where $k < d$, has to be specified *a priori*.

Initialization: $X_0 = Y$, $k = d$

- We initialize the algorithm with the given feature set so that the $k = d$.

Backward Feature Elimination

Wrapper Methods

Step 1 (Exclusion):

$x^- = \arg\max J(x_k - x)$, where $x \in X_k$

$X_{k-1} = X_k - x^-$

$k = k - 1$

Go to Step 2

- In this step, we remove a feature, x^- from our feature subset X_k.
- x^- is the feature that maximizes our criterion function upon re,oval, that is, the feature that is associated with the best classifier performance if it is removed from X_k.

Backward Feature Elimination

Wrapper Methods

Step 2 (Conditional Inclusion):

$$x^+ = \arg\max J(x_k + x), \text{ where } x \in Y - X_k$$

if J(x_k + x) > J(x_k + x):

$$X_{k+1} = X_k + x^+$$

$$k = k + 1$$

Go to Step 1

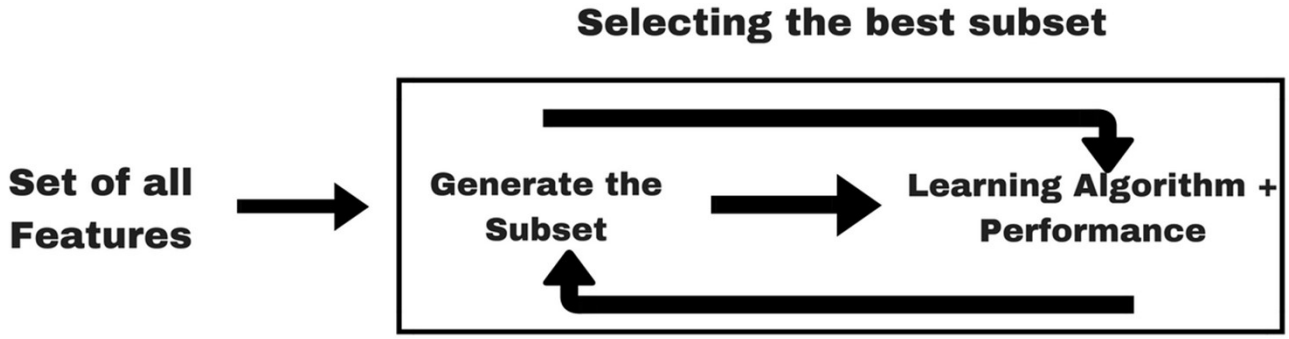
- In Step 2, we search for features that improve the classifier performance if they are added back to the feature subset. If such features exist, we add the feature x^+ for which the performance improvement is maximized. If $k = 2$ or an improvement cannot be made (i.e., such feature x^+ cannot be found), go back to step 1; else, repeat this step.

Termination: $k = p$

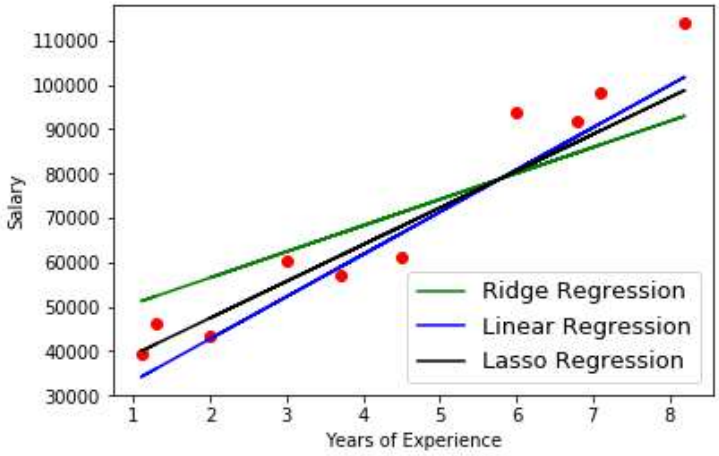
- We add features from the feature subset X_k until the feature subset of size k contains the number of desired features p that we specified *a priori*.

Embedded Methods

- Embedded methods combine the qualities' of filter and wrapper methods. It's implemented by algorithms that have their own built-in feature selection methods.



Embedded Methods



- Some of the most popular examples of these methods are LASSO and RIDGE regression which have inbuilt penalization functions to reduce overfitting.
- Lasso regression performs L1 regularization which adds penalty equivalent to absolute value of the magnitude of coefficients.
- Ridge regression performs L2 regularization which adds penalty equivalent to square of the magnitude of coefficients.

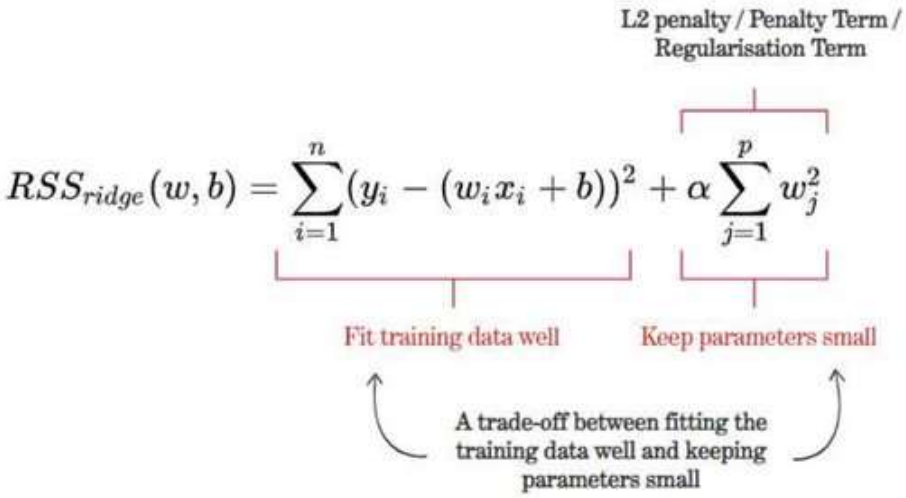
1. **Ridge Regression:**
   - Performs L2 regularization, i.e. adds penalty equivalent to **square of the magnitude** of coefficients
   - Minimization objective = LS Obj + α * (sum of square of coefficients)

2. **Lasso Regression:**
   - Performs L1 regularization, i.e. adds penalty equivalent to **absolute value of the magnitude** of coefficients
   - Minimization objective = LS Obj + α * (sum of absolute value of coefficients)

https://www.youtube.com/watch?v=Q81RR3yKn30

Ridge regression

- Ridge regression also adds an additional term to the cost function, but instead sums the squares of coefficient values (the L-2 norm) and multiplies it by some constant lambda.



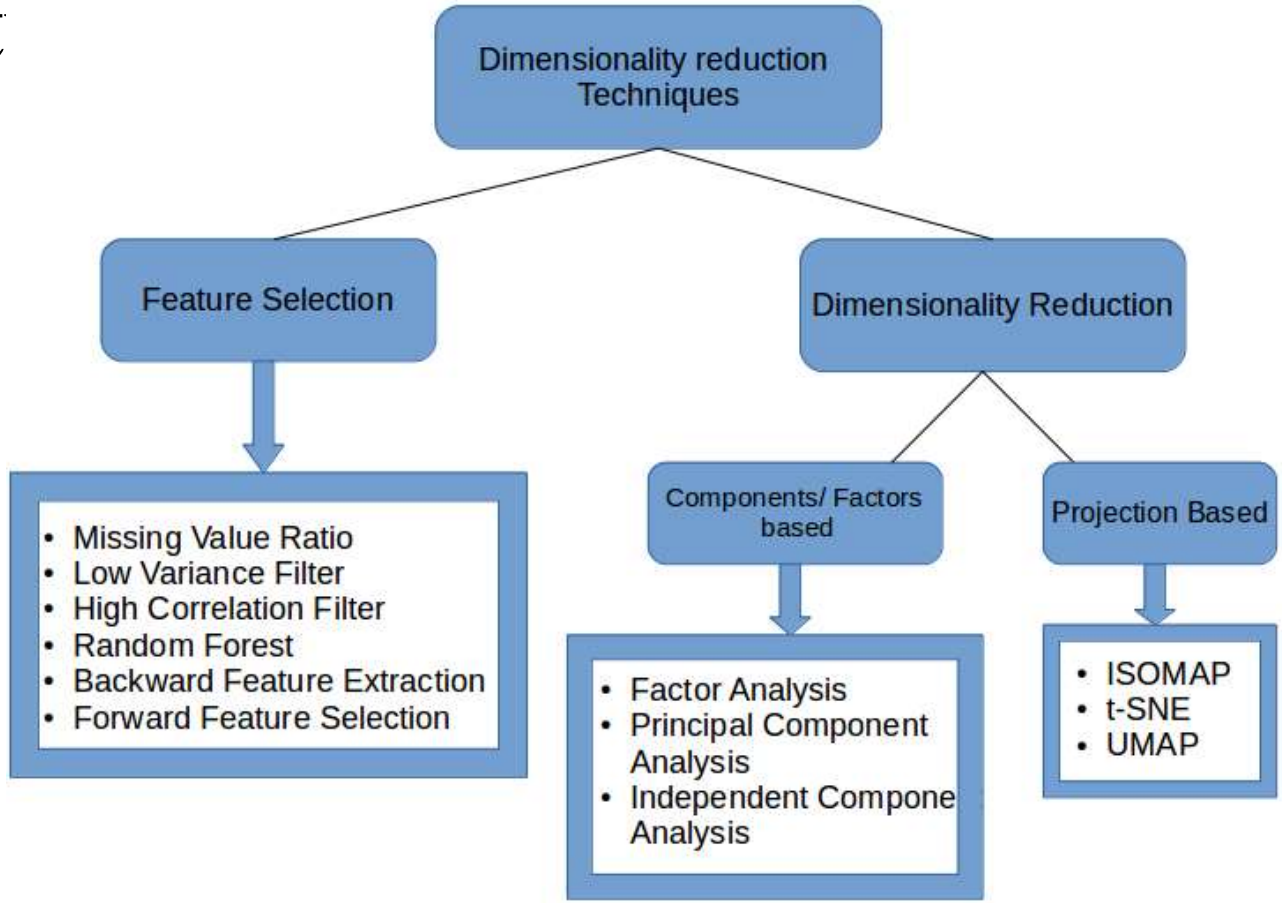
- Compared to Lasso, this regularization term will decrease the values of coefficients, but is unable to force a coefficient to exactly 0. This makes ridge regression's use limited with regards to feature selection. However, when p > n, it is capable of selecting more than n relevant predictors if necessary unlike Lasso. It will also select groups of colinear features, which its inventors dubbed the 'grouping effect.'
- Much like with Lasso, we can vary lambda to get models with different levels of regularization with lambda=0 corresponding to OLS and lambda approaching infinity corresponding to a constant function.

Difference between Filter and Wrapper methods

The main differences between the filter and wrapper methods for feature selection are:

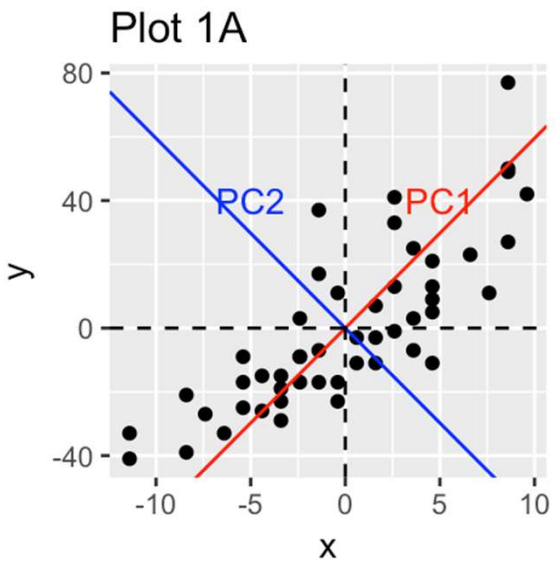
- Filter methods measure the relevance of features by their correlation with dependent variable while wrapper methods measure the usefulness of a subset of feature by actually training a model on it.
- Filter methods are much faster compared to wrapper methods as they do not involve training the models. On the other hand, wrapper methods are computationally very expensive as well.
- Filter methods use statistical methods for evaluation of a subset of features while wrapper methods use cross validation.
- Filter methods might fail to find the best subset of features in many occasions but wrapper methods can always provide the best subset of features.
- Using the subset of features from the wrapper methods make the model more prone to overfitting as compared to using subset of features from the filter methods.

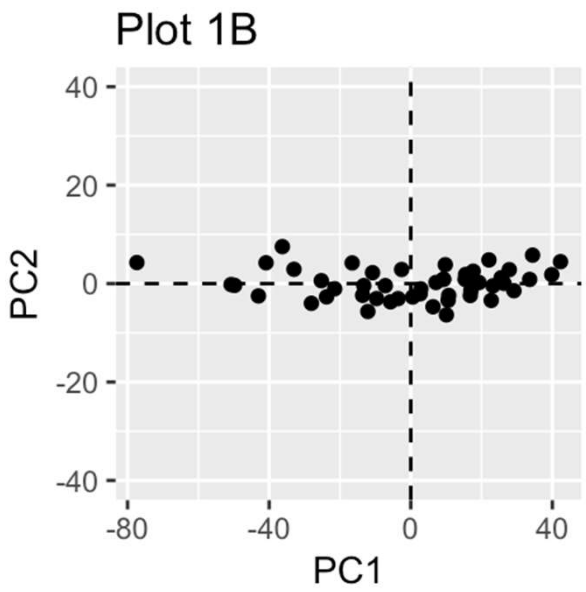
Dimensionalit



Principal Component Analysis

- **Principal component analysis** is used to extract the important information from a multivariate data table and to express this information as a set of few new variables called **principal components**. These new variables correspond to a linear combination of the originals. The number of principal components is less than or equal to the number of original variables.





Principal Component Analysis

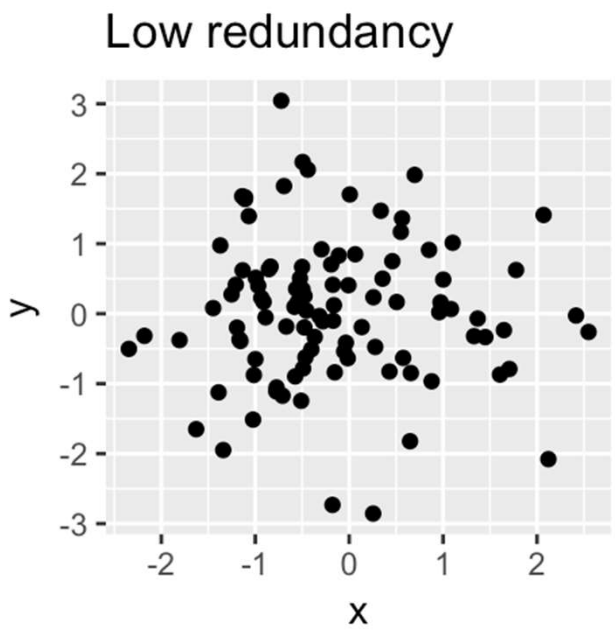
- The information in a given data set corresponds to the total variation it contains. The goal of PCA is to identify directions (or principal components) along which the variation in the data is maximal.
- In other words, PCA reduces the dimensionality of a multivariate data to two or three principal components, that can be visualized graphically, with minimal loss of information.
- Technically speaking, the amount of variance retained by each principal component is measured by the so-called **eigenvalue**.

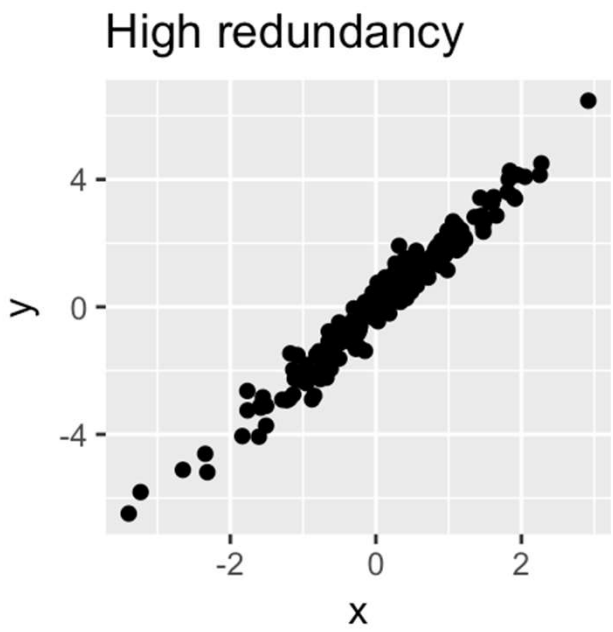
Taken together, the main purpose of principal component analysis is to:

- identify hidden pattern in a data set,
- reduce the dimensionnality of the data by removing the noise and redundancy in the data,
- identify correlated variables

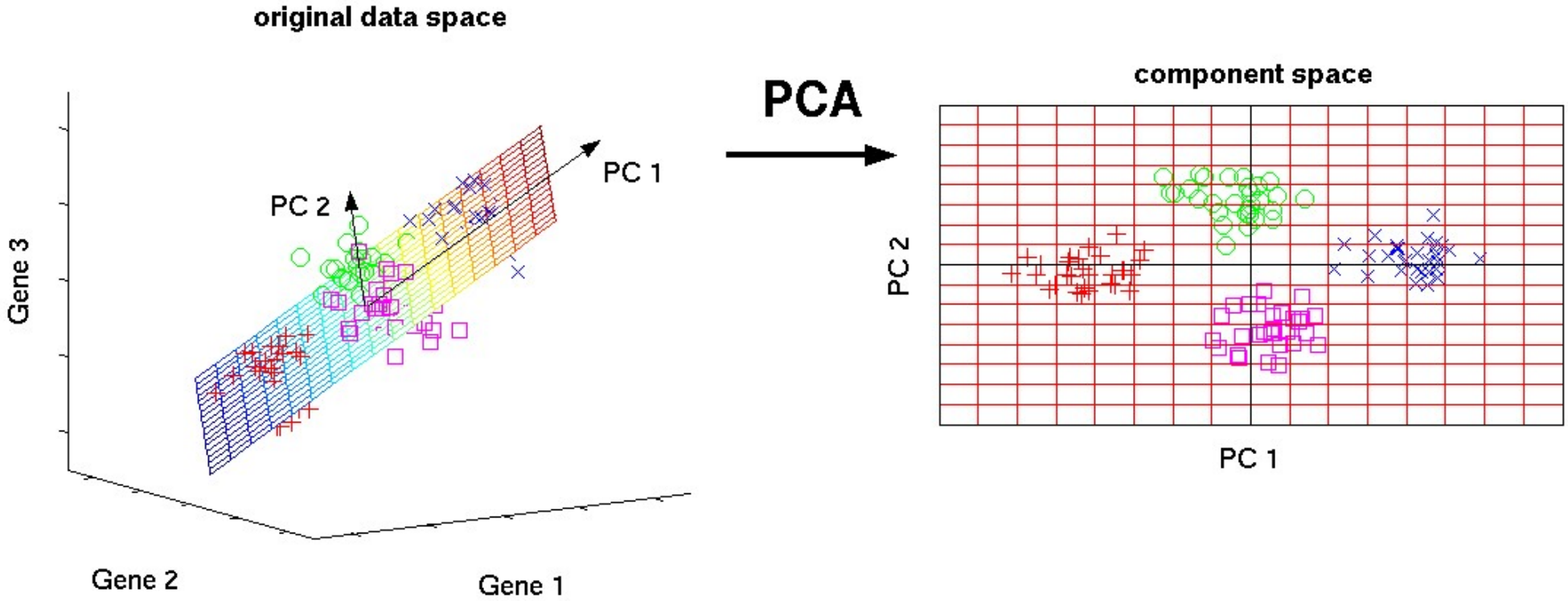
Principal Component Analysis

- Note that, the PCA method is particularly useful when the variables within the data set are highly correlated. Correlation indicates that there is redundancy in the data. Due to this redundancy, PCA can be used to reduce the original variables into a smaller number of new variables (= **principal components**) explaining most of the variance in the original variables.





Principal Component Analysis



1. Compute the mean feature vector

$$\mu = \frac{1}{p}\sum_{k=1}^{p} x_k$$, where, x_k is a pattern ($k = 1$ to p), p = number of patterns, x is the feature matrix

2. Find the covariance matrix

$$C = \frac{1}{p}\sum_{k=1}^{p}\{x_k - \mu\}\{x_k - \mu\}^T$$ where, T represents matrix transposition

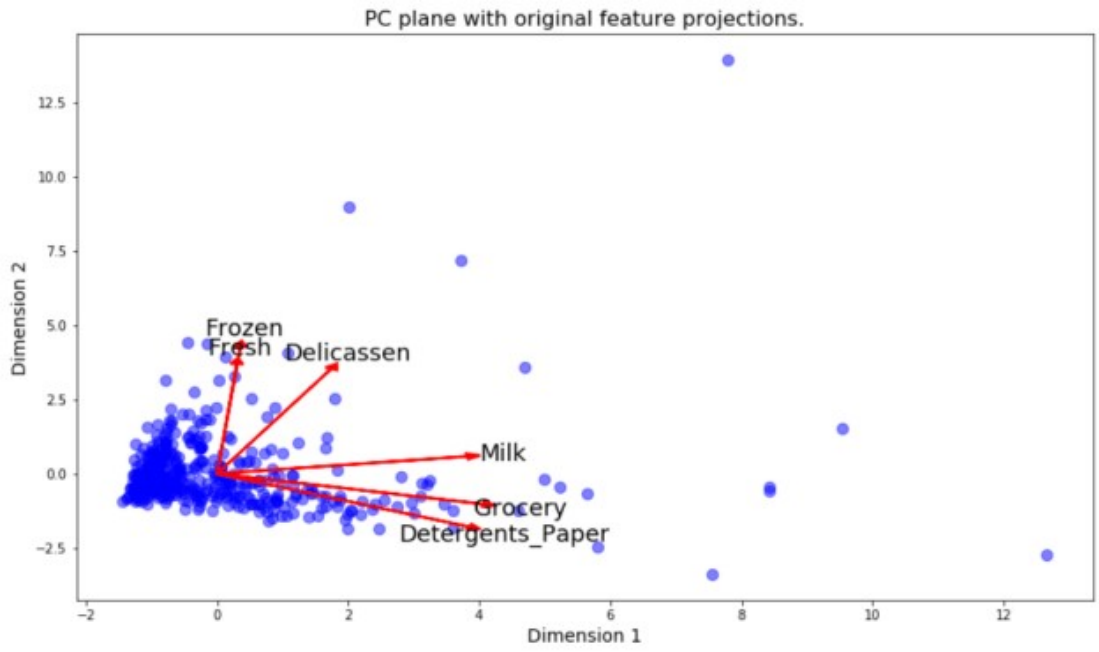
3. Compute Eigen values λ_i and Eigen vectors v_i of covariance matrix
   $Cv_i = \lambda_i v_i$ ($i = 1, 2, 3, ...q$), q = number of features
4. Estimating high-valued Eigen vectors
   (i) Arrange all the Eigen values (λ_i) in descending order
   (ii) Choose a threshold value, θ
   (iii) Number of high-valued λ_i can be chosen so as to satisfy the relationship

$$\left(\sum_{i=1}^{s}\lambda_i\right)\left(\sum_{i=1}^{q}\lambda_i\right)^{-1} \geq \theta$$, where, s = number of high valued λ_i chosen

   (iv) Select Eigen vectors corresponding to selected high valued λ_i
5. Extract low dimensional feature vectors (principal components) from raw feature matrix.
   $P = V^T x$, where, V is the matrix of principal components and x is the feature matrix

Principal Component Analysis

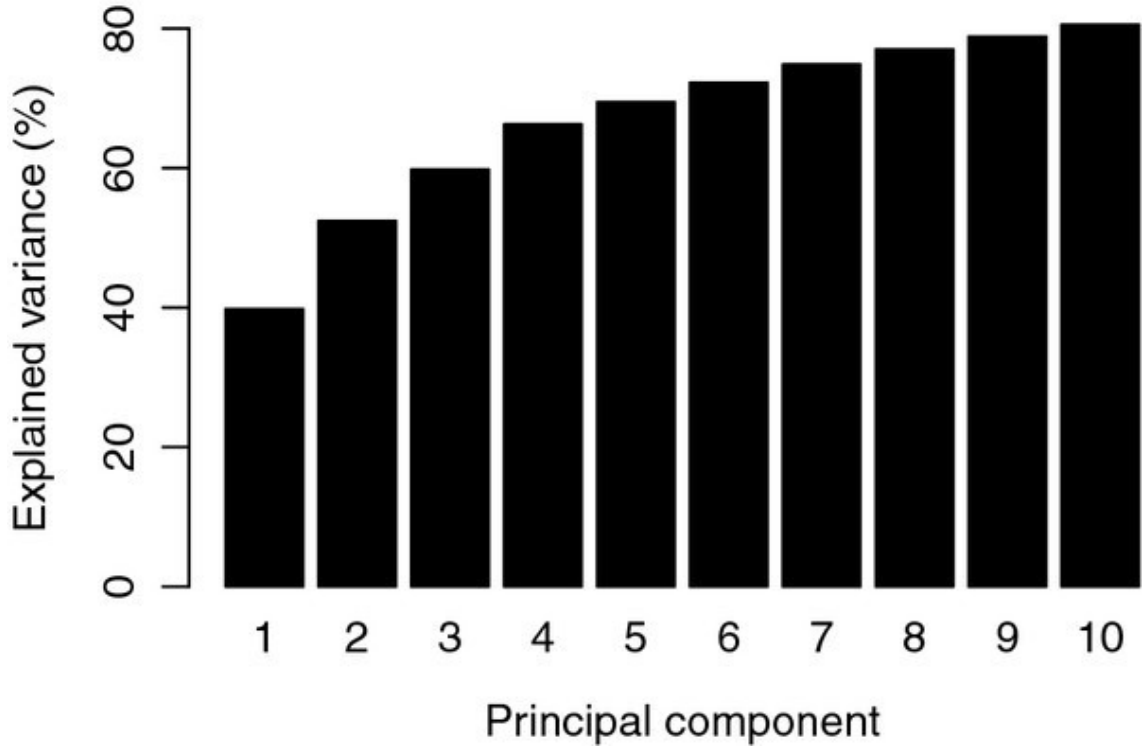
The biplot above shows that the products milk, grocery, and detergents_paper are aligned towards the principal component 1 or dimension 1. Whereas the fresh and frozen products are aligned towards the principal component 2 or dimension 2. These seem intuitive as we have already seen their relationship in the scatter plot above where there seems to be a linear relationship between the group of products milk, grocery and detergents_paper and fresh and frozen products.



Biplot

Principal Component Analysis

The cumulative explained variance in the principal component analysis of the evolutionary distance matrix. After three components more than 60% of the total variance is captured.



THANK YOU

Dr. Tran Anh Tuan, Faculty of Mathematics and Computer Science, University of Science, HCMC